

# claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_3512746c-54e\agent-af86ae8a015599808.jsonl`  
- SHA-256: `9f839ecd6aafdd98f0bdba7b05efd12b6a08b2ba133778096844fb7aeb224e2d`  
- Source modified: `2026-07-06T09:14:33+00:00`  
- Imported at: `2026-07-08T16:00:05+00:00`  
- project: `wf_3512746c-54e`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

## Transcript

### 1. user (2026-07-06T09:13:45.214Z)

Reviewing GitHub PR #39 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-e-selective-scan -> main, one commit db87cc0).  
It implements ADR 0009 Track E: selective source scanning ? /watch, /unwatch, /channels, /watchlist commands +  
an ingestion gate in telethon\_client.py that skips unwatched sources. Changed files ONLY:  
command\_service.py (new /watch etc + \_resolve\_source), commands.py (re-export patterns), db.py (set\_source\_watched, list\_account\_sources), telethon\_client.py (the gate),  
tests/test\_command\_service.py (10 tests).  
Diff +254/-3. READ docs/adr/0009-selective-channel-scanning.md ? it is the spec this must conform to.

KEY ADR 0009 REQUIREMENTS (quote/verify against the code):

- Line 37-38: "The default is locked as opt-in: DEFAULT 0, so a freshly-onboarded user scans nothing until they opt in."
- Line 44: "...return early if watched is false, before any dedup or filter work."
- Line 130-131: "New channels discovered after the migration default to watched = 0 and must be explicitly watched."
- Line 48-54 (discoverability): lazy-populated rows should be created watched=0 so they appear in /channels for opt-in;  
/channels should also upsert candidates from client.get\_dialogs().
- Line 64: "/watch <#name|id>" ? resolve by channel name/username OR id.

Ground rules:

- READ actual code (Read/Grep), cite file:line. You MAY run repros with C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe (build schema via db.\_apply\_schema on aiosqlite ":memory:"; seed via db.upsert\_account + db.upsert\_source; drive command\_service.CommandService().handle()).  
NOTE: real telegram sources have a NUMERIC provider\_source\_id (str(channel\_id) e.g. '-1001234567890'); upsert\_channel  
creates them with watched=True. The on\_new\_message gate is inside a '# pragma: no cover' function.
- Authoritative gate ALREADY RUN (don't re-run full suite): ruff check + format clean; pytest 492 passed @ 97.10% (floor 80%).
- Report ONLY defects in THIS PR's diff. Deviation from the ACCEPTED ADR 0009 is a high-value finding. No praise.  
Concrete failure scenario per finding, ranked most-severe first, honest severity.
- Severity: blocker (feature broken / data loss on the live path), high (real bug or ADR-contract violation users hit),  
medium (edge/latent/half-broken feature), low (hygiene/UX/discoverability).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read the cited code AND docs/adr/0009-selective-channel-scanning.md, trace real control flow, run a repro with

C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe if useful. CONFIRMED only if the defect genuinely ships in THIS PR's diff  
 and (for ADR-conformance) actually deviates from the accepted ADR. REFUTED if the code conforms / trigger can't occur /  
 pre-existing. PLAUSIBLE if unsettled. Corrected severity.  
 FINDING (adr\_gate): /watch cannot resolve a source by channel name/username ? only numeric id (ADR 0009 line 64)  
 severity low | src/tg\_video\_filter/command\_service.py:402  
 summary: ADR 0009 line 64 specifies `/watch <#name|id>` ? resolve by channel name/username OR id. `\_resolve\_source` only looks up `provider\_source\_id`, which holds the numeric Telegram channel\_id (models.py:136-138, upsert\_channel maps str(channel\_id)). It strips a leading '@' and then queries the numeric-id column, so an '@username' or a human channel name never matches unless the user types the raw numeric id.  
 scenario: A user runs `/watch @SpiritualMusic` or `/watch Spiritual Music`. `\_resolve\_source` strips '@' to 'SpiritualMusic', calls get\_source\_by\_provider\_id with that string against the provider\_source\_id column (which stores e.g. '-1001234567890'), gets None, then retries with '@SpiritualMusic' (also None). The user gets 'Source not found' even though the channel is a known source. Only `/watch -1001234567890` works, forcing users to know raw numeric channel ids ? contrary to the ADR's `<#name|id>` affordance.  
 evidence: command\_service.py:410-416: target.lstrip('@') then two lookups both against provider\_source\_id via get\_source\_by\_provider\_id; no lookup against title/username. models.py:136-138 documents provider\_source\_id as the numeric 'Telegram channel\_id'; db.py:1399 upsert\_channel stores str(channel.channel\_id). No username/title column is queried. ADR 0009 line 64: '/watch <#name|id> ? set watched=1 for a source channel' with the discoverability note framing sources by name.  
 rec: Extend \_resolve\_source to also match by title/username (e.g. a case-insensitive LIKE/exact match on sources.title, and store/resolve the @username where available) so a user can /watch by name as the ADR specifies, falling back to numeric id. If username resolution requires the live client, resolve via the Telethon adapter and pass the resulting numeric id into the service.

## 2. assistant (2026-07-06T09:13:47.276Z)

I'll verify this finding by reading the actual code and the ADR.

## 3. user (2026-07-06T09:13:48.067Z)

```

1      # ADR 0009: Selective channel scanning and multiplexed routing
2
3      - **Status**: Accepted
4      - **Date**: 2026-07-04
5      - **Decision owner**: project owner
6
7      ## Context
8
9      Today the userbot scans every channel/group the account is a member of.
10     `manager.py` registers `events.NewMessage(incoming=True)` with no `chats=`
11     filter, and `on_new_message` processes every video regardless of source. The
12     `channels` table exists but is populated lazily as a record of seen channels;
13     it is not used as a filter.
14
15     The product vision is a multiplexing video re-distribution system:
16
17     - A small set of high-quality source channels (e.g. spiritual, classical
18       music) are scanned for incoming videos.
19     - Matches are routed to themed destination broadcast channels (e.g. a
20       Reggae channel, a Rap channel) that subscribers opt into.
21     - A user therefore subscribes to outputs by topic, not to inputs by source.
22
23     This requires two capabilities the current architecture does not have:
24
25     1. Selective scanning ? choose which source channels are watched, rather
26       than ingesting all of them.
27     2. Per-source ? per-destination routing ? a match from source channel X is
28       delivered to a specific destination channel Y, instead of the single global
29       `delivery_target` (ADR 0005).
```

```

30
31 ## Decision
32
33 Add three additive, independently-shippable capabilities:
34
35 ### 1. Selective scanning (watched-flag)
36
37 Add a `watched` flag to the `channels` table. The default is locked as
38 **opt-in: `DEFAULT 0`**, so a freshly-onboarded user scans nothing until they
39 explicitly `/watch` a channel. This matches the multiplexing product shape
40 (sources are a *curated* set) and avoids blindly re-ingesting every chat the
41 account is a member of.
42
43 Gate `on_new_message` at the top: after computing `channel_id`, look up the
44 row and `return` early if `watched` is false, before any dedup or filter work.
45
46 This is a small migration plus ~5 lines in `telethon_client.py`.
47
48 **Discoverability note.** With `DEFAULT 0` plus the current lazy population
49 (a `channels` row is only inserted when a message is first seen), opt-in is
50 necessary but not sufficient for discoverability: a channel the account has
51 never received a message from has no row to flag. The `/channels` command
52 (step 2 below) therefore upserts candidate rows from `client.get_dialogs()`
53 so users can browse and watch channels they have not yet received traffic
54 from. The flag default and the enumeration flow are orthogonal ? both are
55 required.
56
57 ### 2. Channel enumeration and management commands
58
59 New Saved Messages commands (and, via ADR 0008, bot commands) building on the
60 existing `commands.py` pattern:
61
62 - `/channels` ? list the user's channels (via `client.get_dialogs()`), already
63   used in `backfill.py`) with their watched state.
64 - `/watch <#name|id>` ? set `watched=1` for a source channel.
65 - `/unwatch <#name|id>` ? set `watched=0`.
66 - `/watchlist` ? list only watched sources.
67
68 ### 3. Per-source ? per-destination routing rules
69
70 Introduce a `routing_rules` table (the canonical name; issue #25's `routes`
71 refers to the same concept and should be aligned to `routing_rules`), e.g.:
72
73 ```
74 routing_rules (
75     user_id          INTEGER NOT NULL REFERENCES users(id),
76     source_channel_id INTEGER NOT NULL,
77     dest_channel_id   INTEGER NOT NULL,    -- a broadcast channel / @username
78     filter_config_id  INTEGER,             -- optional per-route filter override
79     PRIMARY KEY (user_id, source_channel_id, dest_channel_id)
80 )
81 ```
82
83 On a match, look up applicable routing rules by `source_channel_id` and
84 deliver to each matching destination. The delivery layer (`delivery.py`)
85 already accepts a `target` per call, so this generalises the existing single
86 `delivery_target` to a per-match lookup. Users with no routing rules fall
87 back to the current global `delivery_target` behaviour (ADR 0005), preserving
88 backwards compatibility.
89
90 ## Consequences
91
92 **Positive**
93 - Enables the multiplexing product shape: scan curated sources, publish to
94   topical broadcast channels, let subscribers pick outputs by interest.

```

```

95 - Each capability is additive and independently deployable; none disturbs the
96 existing FORWARD/LINK/BOTH paths or the metadata-first ingestion (ADR 0003).
97 - Dedup (ADR 0006) still holds: Tier 1 (`document_id`) keys the *original*
98 file, so a routed re-upload is not re-processed.
99 - Channel enumeration reuses `client.get_dialogs()` already present in
100 `backfill.py`; no new Telethon capability is required.
101
102 **Negative**
103 - New schema (`channels.watched`, `routing_rules`) with migrations.
104 - **Routing loop invariant.** With multiplexed routing, the userbot account
105 may be a member of a *destination* broadcast channel and re-see its own
106 output, producing an infinite forward loop. The contract is: **a channel
107 is either a source or a destination, never both.** This is enforced in
108 `on_new_message` by excluding any channel that is a configured destination
109 (`source_channel_id IN (SELECT dest_channel_id FROM routing_rules WHERE
110 user_id = ?)`) before the watched-flag check. Issue #25 Track E implements
111 this; the invariant is stated here so the handler has a contract to code
112 against.
113 - Per-route filter overrides add config complexity; v1 may ship with global
114 filters only and a single implicit route (`source ? global target`), then
115 layer explicit per-route rules later.
116
117 ## Backwards compatibility
118
119 Adding `channels.watched` with `DEFAULT 0` would silently stop every existing
120 install from receiving matches until the user re-watches their channels ? a
121 behaviour change disguised as a schema change. To preserve the current
122 "ingest everything" semantics for existing installs, the migration backfills:
123
124 ```sql
125 ALTER TABLE channels ADD COLUMN watched INTEGER NOT NULL DEFAULT 0;
126 UPDATE channels SET watched = 1 WHERE seen_first_at IS NOT NULL;
127 ```
128
129 i.e. any channel the install has *already* seen traffic from is implicitly
130 watched, so pre-migration users observe no change. New channels discovered
131 after the migration default to `watched = 0` and must be explicitly watched ?
132 which is the intended new UX. This mirrors the "old rows behave exactly as
133 before" pattern established by ADR 0005.
134
135 ## Rollout order
136
137 1. **Selective scanning** (`channels.watched` + `/watch`, `/unwatch`,
138 `/channels`). Smallest unit; immediately useful even without routing.
139 2. **Named destination channels** ? extend `/set delivery_target` to allow
140 switching among a user-defined set, not just one global value.
141 3. **Per-source ? per-destination routing rules** (`routing_rules` table +
142 `/route <source> <dest>` command). The full multiplexing vision.
143
144 Each step is shippable and useful on its own; step 3 is not required to
145 deliver value from steps 1?2.
146
147 ## Alternatives considered
148
149 - **Server-side `chats=` filter at handler registration:** rejected ? the
150 watched set is per-user and mutable at runtime; re-registering Telethon
151 handlers on every `/watch` change is brittle. An in-handler lookup against
152 the `channels` table is simpler and hot-swappable.
153 - **Content-based classification (auto-detect genre):** explicitly out of
154 scope for v1. Routing is user-defined by source channel; automatic topic
155 detection could be a future enhancement but is not required to deliver the
156 multiplexing UX.
157
158 ## References
159

```

```

160 - ADR 0001 (userbot ingestion) ? the capability this selectively constrains.
161 - ADR 0005 (per-user delivery target) ? the single-target delivery this
162 generalises into per-route targets.
163 - ADR 0006 (content dedup) ? still applies; routed re-uploads are keyed by
164 the original `document_id`.
165 - ADR 0008 (bot API onboarding) ? the front-door that exposes `/watch`,
166 `/unwatch`, and routing commands to public users.
167

```

#### 4. user (2026-07-06T09:13:48.890Z)

```

380         lines = ["? **All sources**", ""]
381         for s in sources:
382             icon = "?" if s.watched else "?"
383             label = s.title or s.provider_source_id
384             lines.append(f"{icon} `{s.provider_source_id}` ? {label}")
385         return CommandResult("\n".join(lines))
386
387     async def _handle_watchlist(self, conn: aisqlite.Connection, user_id: int) ->
CommandResult:
388         """List only watched sources."""
389         sources = await db.list_account_sources(conn, user_id, watched_only=True)
390         if not sources:
391             return CommandResult(
392                 "? **No watched sources.**\n\n"
393                 "Use `/channels` to see available sources, then `/watch <id>` "
394                 "to start scanning."
395             )
396         lines = ["? **Watched sources**", ""]
397         for s in sources:
398             label = s.title or s.provider_source_id
399             lines.append(f"? `{s.provider_source_id}` ? {label}")
400         return CommandResult("\n".join(lines))
401
402     async def _resolve_source(
403         self, conn: aisqlite.Connection, user_id: int, target: str
404     ) -> Source | None:
405         """Resolve a user-provided source identifier to a ``Source`` row.
406
407         Accepts ``@username`` or a numeric channel id string. The ``@`` prefix
408         is stripped before lookup since Telegram channel ids are numeric.
409         """
410         target = target.lstrip("@")
411         # Try by provider-native id first (the common case).
412         src = await db.get_source_by_provider_id(conn, user_id, "telegram", target)
413         if src is not None:
414             return src
415         # Fall back: try with @ prefix in the target column (just in case).
416         return await db.get_source_by_provider_id(conn, user_id, "telegram",
f"@{target}")
417
418     # -- internal -----
419
420     async def _update(
421         self,
422         conn: aisqlite.Connection,
423         user_id: int,
424         current: FilterConfig,
425         max_size_mb: int | None = None,
426         max_duration_s: int | None = None,
427         min_duration_s: int | None = None,
428         target_height: int | None = None,
429         exclude_gif: bool | None = None,
430         exclude_round: bool | None = None,
431         exclude_nosound: bool | None = None,

```

```

432 ) -> FilterConfig:
433     """Apply partial updates to the account's config, persist, and return."""
434     updates: dict[str, object] = {}
435     if max_size_mb is not None:
436         updates["max_size_mb"] = max_size_mb
437     if max_duration_s is not None:
438         updates["max_duration_s"] = max_duration_s
439     if min_duration_s is not None:
440         updates["min_duration_s"] = min_duration_s
441     if target_height is not None:
442         updates["target_height"] = target_height
443     if exclude_gif is not None:
444         updates["exclude_gif"] = exclude_gif
445     if exclude_round is not None:
446         updates["exclude_round"] = exclude_round
447     if exclude_nosound is not None:
448         updates["exclude_nosound"] = exclude_nosound
449     cfg = current.model_copy(update=updates) if updates else current
450     await db.save_filter_config(conn, user_id, cfg)
451     log.info(
452         "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
453         "target_height=%s exclude_gif=%s exclude_round=%s exclude_nosound=%s",
454         user_id,
455         cfg.max_size_mb,
456         cfg.min_duration_s,
457         cfg.max_duration_s,
458         cfg.target_height,
459         cfg.exclude_gif,

```

## 5. assistant (2026-07-06T09:13:53.395Z)

Now let me verify how sources are created and whether a username/title column exists. Let me check the db functions and models.

## 6. user (2026-07-06T09:13:54.432Z)

```

120 # Option 2A: ``policies`` generalizes ``filter_config``.
121
122
123 class Source(BaseModel):
124     """A configured place we ingest content from (ADR 0011).
125
126     Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future
127     providers add their own source kinds). ``provider_source_id`` is the
128     provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
129     """
130
131     model_config = ConfigDict(frozen=True)
132
133     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
134     account_id: int = Field(description="FK ? accounts.id (owner of the source)")
135     provider: str = Field(default="telegram", description="Provider namespace")
136     provider_source_id: str = Field(
137         description="Provider-native source id (Telegram channel_id, ?)"
138     )
139     title: str | None = None
140     kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
141     watched: bool = Field(
142         default=False,
143         description="ADR 0009 selective scanning: ingest from this source iff True",
144     )
145     seen_first_at: datetime = Field(default_factory=_utcnow)
146     last_seen_at: datetime | None = None
147
148

```

```

149     class ContentItem(BaseModel):
150         """A normalized content record discovered from any :class:`Source` (ADR 0011).
151
152         Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
153         (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
154         NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
155         ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
156         wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
157         """
158
159         model_config = ConfigDict(frozen=True)

```

## 7. user (2026-07-06T09:13:55.695Z)

```

1142:async def get_source_by_provider_id(
1143-     conn: aiosqlite.Connection, account_id: int, provider: str, provider_source_id: str
1144-) -> Source | None:
1145-     """Resolve a source by its provider-native id."""
1146-     conn.row_factory = aiosqlite.Row
1147-     async with conn.execute(
1148-         """SELECT * FROM sources
1149-            WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
1150-         (account_id, provider, provider_source_id),
1151-     ) as cur:
1152-         row = await cur.fetchone()
1153-     return _row_to_source(row) if row else None
1154-
1155-
1156-# --- ContentItem repos (ADR 0011) -----
1157-
1158-
1159:async def insert_content_item(conn: aiosqlite.Connection, item: ContentItem) ->
ContentItem:
1160-     """Insert a content item. Raises ``IntegrityError`` on a dedup collision.
1161-
1162-     The ``dedup_key`` must be precomputed by :func:`compute_dedup_key`. Two
1163-     unique constraints guard against duplicates:
1164-     - ``UNIQUE (account_id, source_id, provider_message_id)`` ? Tier-0.
1165-     - ``UNIQUE (account_id, dedup_key)`` ? Tier-1/2 (provider-neutral).
1166-     """
1167-     cur = await conn.execute(

```

## 8. assistant (2026-07-06T09:13:59.510Z)

The finding is confirmed on the mechanics: ``\_resolve\_source`` only queries ``provider\_source\_id``. Now the key question for severity: is resolving by name/username actually implementable in THIS PR's context, and does the ADR truly require it? Let me check the ``title`` column and how sources get titles. Let me also verify the finding is about THIS PR's diff.

## 9. user (2026-07-06T09:14:01.091Z)

```

59:     title          TEXT,
97:     title          TEXT,
149:    title          TEXT,
441:    title          TEXT,
446:    "id, user_id, channel_id, title, seen_first_at",
522:    # 1a. Backfill sources from channels first (known title + watched=1). Order
523:    #    matters: channels-first so title/watched data wins; the videos-based
528:    (account_id, provider, provider_source_id, title, kind, watched,
530:    SELECT user_id, 'telegram', CAST(channel_id AS TEXT), title, 'channel',
539:    #    ? the channels-based row above survives with its title + watched=1.
849:    title=row["title"],
899:    title=row["title"],
1105:    """Insert a source row, or update title/watched on conflict.

```

```

1112:         (account_id, provider, provider_source_id, title, kind, watched,
1116:         title=excluded.title,
1123:         source.title,
1289:         (account_id, provider, target, kind, title, is_output, created_at)
1293:         title=excluded.title,
1300:         dest.title,
1400:         title=channel.title,
1432:         title=src.title,

```

#### 10. assistant (2026-07-06T09:14:05.374Z)

There is a `title` column in the `sources` table (schema line 97), and there is no `username` column. Let me look at the sources table schema and check `list\_account\_sources` (a function in this PR's diff) to confirm the shape.

#### 11. user (2026-07-06T09:14:06.376Z)

```

90      -- the delivery logic that populates them.
91
92      CREATE TABLE IF NOT EXISTS sources (
93          id                INTEGER PRIMARY KEY AUTOINCREMENT,
94          account_id        INTEGER NOT NULL REFERENCES accounts(id),
95          provider          TEXT NOT NULL DEFAULT 'telegram',
96          provider_source_id TEXT NOT NULL,
97          title             TEXT,
98          kind              TEXT NOT NULL DEFAULT 'channel',
99          watched           INTEGER NOT NULL DEFAULT 0,
100         seen_first_at     TEXT NOT NULL,
101         last_seen_at      TEXT,
102         UNIQUE (account_id, provider, provider_source_id)
103     );
104
105     CREATE TABLE IF NOT EXISTS content_items (
106         id                INTEGER PRIMARY KEY AUTOINCREMENT,
107         account_id        INTEGER NOT NULL REFERENCES accounts(id),
108         source_id         INTEGER NOT NULL REFERENCES sources(id),
109         provider          TEXT NOT NULL DEFAULT 'telegram',

```

#### 12. user (2026-07-06T09:14:07.123Z)

```

340
341     async def _handle_watch(
342         self, conn: aiomysql.Connection, user_id: int, target: str, _cfg: FilterConfig
343     ) -> CommandResult:
344         """Parse `/watch <@username|channel_id>` and toggle `watched=1`."""
345         src = await self._resolve_source(conn, user_id, target)
346         if src is None:
347             return CommandResult(
348                 f"? Source `{target}` not found. Use `/channels` to list "
349                 f"available sources.\n\nSources are discovered as messages "
350                 f"arrive; a channel you haven't received traffic from may not "
351                 f"appear yet."
352             )
353         await db.set_source_watched(conn, user_id, src.id, True) # type: ignore[arg-
type]
354         label = src.title or src.provider_source_id
355         return CommandResult(f"? Now watching **{label}**"
(f"{src.provider_source_id}").")
356
357     async def _handle_unwatch(
358         self, conn: aiomysql.Connection, user_id: int, target: str, _cfg: FilterConfig
359     ) -> CommandResult:
360         """Parse `/unwatch <@username|channel_id>` and toggle `watched=0`."""
361         src = await self._resolve_source(conn, user_id, target)

```

```

362         if src is None:
363             return CommandResult(
364                 f"? Source `{target}` not found. Use `/channels` to list sources."
365             )
366         await db.set_source_watched(conn, user_id, src.id, False) # type: ignore[arg-
type]
367         label = src.title or src.provider_source_id
368         return CommandResult(f"? Stopped watching **{label}**
({src.provider_source_id}).")
369
370     async def _handle_channels(self, conn: aiomysqlite.Connection, user_id: int) ->
CommandResult:
371         """List all known sources with watch status."""
372         sources = await db.list_account_sources(conn, user_id)
373         if not sources:
374             return CommandResult(
375                 "? **No sources yet.**\n\n"
376                 "Sources are discovered when a new message arrives from a "
377                 "channel. Run `/backfill` to discover historical channels, "
378                 "or wait for new traffic."
379             )

```

### 13. assistant (2026-07-06T09:14:23.850Z)

Now I have a clear picture. Let me assess the finding critically.

Key facts:

- `\_resolve\_source` (command\_service.py:402-416) only queries `provider\_source\_id` (numeric channel\_id), via two lookups both against the same numeric column. It never queries `title`.
- The `sources` table has a `title` column but **no username column**. Titles are human names like "Spiritual Music" (populated from `channel.title` at upsert, db.py:1400).
- ADR 0009 line 64: `/watch <#name|id>` ? the `<#name|id>` affordance implies resolving by name/username OR id.
- The docstring at command\_service.py:407 explicitly claims "Accepts `@username` or a numeric channel id string." ? but there is no username column at all in `sources`, so `@username` resolution can NEVER succeed. It always falls to the numeric-id column.

Let me verify the `\_handle\_channels` upsert-from-dialogs behavior isn't relevant here, and confirm the docstring claim about `@username` is genuinely dead. The critical nuance: does Telegram's `channel.title` ever equal a `@username`? No ? title is the display name. So `/watch @SpiritualMusic` strips `@` ? `SpiritualMusic`, queries numeric-id column ? miss; retries `@SpiritualMusic` ? miss. And `/watch Spiritual Music` ? queries numeric column ? miss.

The finding is mechanically correct. Now ? severity. The reporter says **low**. Let me weigh:

- The ADR's `<#name|id>` is the specified affordance. Resolution by name is genuinely unsupported.
- BUT: the primary path (`/channels` lists sources showing the numeric id in backticks, and `/watch <id>` copy-paste) works. Users are shown the id to copy. So the id path is functional and is the documented UX in the command's own help text (`/watch <id>` at line 393).
- The docstring at :407 makes a **false claim** that `@username` works ? that's a real defect (misleading), but username data isn't even stored, so it's not a regression, just an unfulfilled affordance.

This is a real deviation from the ADR's `<#name|id>` spec, but it's a discoverability/UX shortfall, not a broken live path: `/channels` surfaces the copy-pasteable id and `/watch <id>` works. Name/username resolution being absent is a partial-feature gap. Low is the honest severity ? it's a UX/affordance gap, not a bug users hit on the working path (the working path shows them the id to use).

I'll confirm the finding at severity low.

### 14. user (2026-07-06T09:14:33.537Z)

Structured output provided successfully